

UNIVERSIDAD DEL BÍO-BÍO
FACULTAD DE CIENCIAS EMPRESARIALES
Escuelas de: Ingeniería Civil Informática
Ingeniería En Ejecución En Computación E Informática

Propuesta de Proyecto de Titulación

_____ (fecha)

1 Identificación de los/las estudiantes

1.1 Estudiantes

- **NOMBRE:** Francisco Gonzalo Alexi Moya Alday
- **DIRECCIÓN:** Floresta 2, Pasaje 18, casa 5235
- **TELÉFONO:** +569 2205 3599
- **E-MAIL:** francisco.moya2301@alumnos.ubiobio.cl
- **CARRERA:** Ingeniería En Ejecución En Computación E Informática
- **DEPTO.:** Sistemas de Información

2 Identificación de Guía(s), asesores y supervisión

2.1 Profesor Guía

- **NOMBRE:** Dra. Mónica Caniupán Marileo
- **E-MAIL:** mcaniupan@ubiobio.cl Firma:

2.2 Personas, Instituciones o Empresas en que se solicitará apoyo y/o asesoría

*(**si corresponde; si no, eliminar)*

- **NOMBRE:** _____
- **E-MAIL:** _____
- **RUBRO:** _____ Firma:

3 Proyecto de Título

3.1 Título que identificará la actividad de Titulación

“Optimización de consultas SQL en PostgreSQL mediante técnicas de Machine Learning: selección del orden de JOIN y recomendación de índices a través de un analizador externo desacoplado”

3.2 Modalidad de la propuesta

Desarrollo de Software

Investigación

X Investigación con desarrollo de Software

3.3 Presentación del área y su problemática

3.3.1 Presentación del contexto

El proyecto se enmarca en el área de los Sistemas de Gestión de Bases de Datos (SGBD) relacionales y, en particular, en la optimización de consultas SQL sobre PostgreSQL, uno de los motores de bases de datos de código abierto más utilizados en el ámbito académico y en la industria.

La optimización de consultas es una de las funciones más importantes de un SGBD, ya que determina en gran medida el tiempo de respuesta de las aplicaciones. La selección del orden de ejecución de los JOIN es un problema combinatorio **NP-duro**: el número de órdenes posibles crece de forma factorial con la cantidad de relaciones, por lo que evaluar todas las alternativas es inviable para consultas grandes. A medida que aumentan el volumen de los datos y la complejidad de las consultas, estas decisiones se vuelven más difíciles y más determinantes para el rendimiento, lo que ha motivado el uso de técnicas de **Machine Learning** para apoyar al optimizador.

Alcance conceptual. Este proyecto emplea **Machine Learning clásico y supervisado** (modelos de ranking y regresión basados en árboles, y eventualmente redes neuronales sobre grafos o árboles de plan). No utiliza inteligencia artificial generativa, agentes conversacionales ni Grandes Modelos de Lenguaje (LLM); el término “inteligencia artificial” se reserva únicamente como el campo general que contiene al Machine Learning.

Dado que la modalidad es Investigación con desarrollo de Software, el trabajo se desarrolla en el contexto académico de la carrera de Ingeniería de Ejecución en Computación e Informática de la Universidad del Bío-Bío.

3.3.2 Procesos del ámbito del proyecto

Ante una consulta SQL, el optimizador de PostgreSQL analiza distintas alternativas de ejecución y selecciona un plan que busca usar eficientemente los recursos. Entre las decisiones más relevantes están la selección del orden de ejecución de los JOIN, la elección de los métodos de acceso (recorrido secuencial o por índice) y el uso de índices. Para estimar el costo de cada plan, el optimizador se apoya en estadísticas y en estimaciones de cardinalidad. Las herramientas **EXPLAIN** y **EXPLAIN ANALYZE** permiten inspeccionar el plan elegido, sus operadores, las cardinalidades estimadas, los costos y los tiempos reales de ejecución.

3.3.3 Descripción del problema u oportunidades de mejora

- **Planes subóptimos en consultas complejas:** las estimaciones de cardinalidad pueden fallar y llevar a planes lejos del óptimo, especialmente con muchas relaciones y datos sesgados.
- **Espacio de búsqueda NP-duro para el orden de JOIN:** el número de órdenes posibles crece factorialmente con la cantidad de relaciones.

- **Selección de índices manual:** decidir qué índices crear para un conjunto de consultas requiere experiencia y supone un compromiso entre mejora de rendimiento y espacio/mantenimiento.
- **Falta de diagnóstico de rendimiento:** no siempre es evidente qué consultas sobrecargan la base de datos ni si su lentitud responde a una limitación estructural (por ejemplo, recorridos secuenciales sobre tablas grandes o ausencia de índices) más que al orden de los JOIN.
- **Costo de las soluciones aprendidas existentes:** los optimizadores aprendidos del estado del arte suelen ser intrusivos (modifican el motor) o demandan un entrenamiento muy costoso, lo que dificulta su adopción práctica.

3.4 Propuesta

Se propone diseñar, desarrollar y evaluar un sistema **externo y desacoplado**, basado en técnicas de Machine Learning supervisado, que apoye la optimización de consultas SQL en bases de datos relacionales (con validación en PostgreSQL) en dos problemas complementarios: la selección del orden de ejecución de los JOIN y la recomendación de los mejores índices para un conjunto de consultas (workload). Adicionalmente, el sistema incorpora un análisis del rendimiento de la base de datos —tiempos reales de ejecución y detección de consultas que la sobrecargan o que presentan cuellos de botella—. El sistema se entrega como una plataforma web contenerizada en Docker y no reemplaza al optimizador de PostgreSQL, sino que lo complementa desde fuera del proceso del motor.

3.4.1 Núcleo analítico de Machine Learning

El componente central es un conjunto de modelos de Machine Learning entrenados de forma supervisada:

- **Orden de JOIN:** se plantea como un problema de *aprendizaje de ranking* (learning-to-rank) o de regresión sobre planes candidatos. El modelo estima el tiempo real de ejecución de cada permutación de JOIN y se selecciona la de menor tiempo esperado.
- **Recomendación de índices:** se aborda como la búsqueda del **conjunto de índices más beneficioso (óptimo)** para el workload, evaluando índices candidatos con índices hipotéticos (HypoPG) y un análisis de costo global bajo una restricción de espacio, de modo de maximizar la mejora total minimizando el costo de mantenimiento.
- **Diagnóstico y análisis de rendimiento:** de forma complementaria, el sistema mide y analiza los tiempos reales de ejecución del workload para identificar las consultas más costosas (las que sobrecargan la base de datos) y para distinguir cuándo la lentitud responde a una limitación estructural (cuellos de botella, como recorridos secuenciales o falta de índices) en lugar de al orden de los JOIN.
- **Características (features):** vectores numéricos y categóricos extraídos del plan sintáctico EXPLAIN (ANALYZE, FORMAT JSON): operadores de JOIN (*Hash Join*, *Merge Join*, *Nested Loop*), cardinalidades estimadas, anchos de fila y páginas de disco, entre otros.
- **Variable objetivo (target):** el tiempo real de ejecución en milisegundos (**Actual Total Time**).
- **Modelos:** algoritmos basados en árboles como LightGBM o XGBoost y, de ser necesario, redes neuronales sobre la estructura de árbol/grafos del plan.

3.4.2 Arquitectura de software desacoplada

La solución se despliega como una plataforma web contenerizada en Docker, con tres componentes:

- **Interfaz web (dashboard analítico):** visualización de los árboles de consulta, de las métricas de ejecución y de las sugerencias de índices en formato DDL.
- **Servidor de análisis externo (backend en Python):** aloja los modelos de Machine Learning entrenados y el motor de inferencia. Todo el cómputo matemático ocurre en este servidor independiente.
- **Capa de conexión y protocolo de ingesta (conector de solo lectura / MCP — Model Context Protocol):** se conecta a cualquier servidor PostgreSQL remoto (local, nube o VPS) únicamente para realizar consultas de solo lectura a los catálogos (`pg_catalog`, `information_schema`) y capturar planes EXPLAIN.

Garantía de no impacto: la base de datos objetivo no ejecuta software analítico, no gasta CPU/RAM en inferencia y no queda expuesta a riesgos de concurrencia o de escritura, ya que solo atiende lecturas ligeras de metadatos y planes.

Extensibilidad a múltiples bases de datos relacionales. La capa de conexión (MCP/adaptadores) se diseña de forma agnóstica, de modo que el sistema pueda conectarse a distintos SGBD relacionales remotos. No obstante, la *validación experimental* de esta propuesta se acota a PostgreSQL, ya que el modelo de costos, los planes y las extensiones utilizadas (por ejemplo, HypoPG) son específicos de cada motor; el soporte a otros motores queda planteado como una extensión natural de la arquitectura.

3.5 Análisis de los Principales Trabajos Realizados en el área o tema de la propuesta

La optimización de consultas asistida por aprendizaje es un área activa de investigación. Los trabajos más representativos son:

- **Neo** (Marcus et al., 2019): optimizador aprendido de extremo a extremo que reemplaza por completo la toma de decisiones del motor mediante redes neuronales; es un enfoque potente pero *invasivo* y requiere miles de horas de entrenamiento.
- **Bao** (Marcus et al., 2021): guía al optimizador de PostgreSQL mediante *hints* a nivel de sesión, usando redes convolucionales sobre árboles (Tree-CNN) y muestreo de Thompson; es más práctico que Neo, pero necesita soporte intrusivo en el motor.
- **Lero** (Zhu et al., 2023): plantea la optimización como un problema de *Learning-to-Rank* (LTR) sobre los planes candidatos generados por el SGBD.
- **Balsa** (Yang et al., 2022): optimizador aprendido que se pre-entrena a partir de las estimaciones del modelo de costos, sin ejecutar consultas reales al inicio.
- **AutoAdmin / Database Tuning Advisor** (Chaudhuri y Narasayya, 1997): trabajo clásico de selección de índices dirigida por costo, que introduce el análisis “what-if”.
- **HypoPG** y **pg_qualstats**: herramientas que permiten crear índices hipotéticos y evaluar su beneficio sobre PostgreSQL sin materializarlos.

Diferenciador de este proyecto. A diferencia de los enfoques anteriores, se propone una arquitectura **externa y desacoplada** que no reemplaza ni modifica el optimizador de PostgreSQL. El

sistema combina un modelo de ranking para la selección del orden de JOIN con un evaluador de costo global del workload para la recomendación de índices hipotéticos (**HypoPG**), operando fuera del proceso del motor de base de datos. Esto reduce el costo de adopción, evita la intrusividad y no sobrecarga la base de datos de producción. La Tabla 1 resume esta comparación.

Trabajo / Enfoque	Técnica utilizada	Intrusividad en el SGBD	Sobrecarga en la BD de producción	Alcance de decisión
Neo (2019)	Red neuronal end-to-end	Alta (reemplaza el optimizador)	Alta (entrenamiento costoso)	Orden de JOIN y operadores
Bao (2021)	Tree-CNN + Thompson Sampling (hints)	Media (soporte en el motor)	Media	Selección de plan vía hints
Lero (2023)	Learning-to-Rank	Media	Media	Orden de JOIN (ranking de planes)
Balsa (2022)	RL pre-entrenado con modelo de costos	Media	Baja-Media	Orden de JOIN
AutoAdmin / HypoPG	Heurística por costo / índices hipotéticos	Baja	Baja	Índices
Esta propuesta	ML supervisado externo (LightGBM/XGBoost)	Nula (de-sacoplada)	Mínima (solo lecturas EXPLAIN)	Orden de JOIN + índices (workload)

Tabla 1: Comparación de la propuesta con los principales trabajos del estado del arte.

3.5.1 Delimitación conceptual: Machine Learning vs. Protocolo MCP

Durante la evaluación surgió una confusión frecuente entre el modelo de Machine Learning y el protocolo MCP. Es importante aclarar que **no son lo mismo ni cumplen el mismo rol**: el modelo de Machine Learning es el *cerebro* analítico que aprende y decide, mientras que el MCP es únicamente una *capa de transporte* de solo lectura para conectarse a la base de datos. La Tabla 2 detalla la diferencia.

Aspecto	Modelo de Machine Learning (núcleo de la tesis)	Servidor MCP (Model Context Protocol)
Naturaleza	Algoritmo matemático (LightGBM/XGBoost)	Estándar de software de transporte cliente-servidor (I/O)
¿Aprende / entrena?	Sí, con TPC-H y JOB	No
¿Toma decisiones?	Sí; predice costos/tiempos y recomienda	No; solo transporta información
Rol en el sistema	Inferencia y ranking de planes; recomendación de índices	Conector de solo lectura para extraer metadatos (<code>pg_catalog</code>) y planes EXPLAIN
Ubicación	Servidor de análisis externo (backend)	Capa de conexión hacia el PostgreSQL remoto

Tabla 2: Delimitación conceptual entre el modelo de Machine Learning y el protocolo MCP.

3.6 Justificación del Proyecto

La optimización de consultas es una función crítica en cualquier SGBD, y los optimizadores tradicionales pueden producir planes ineficientes cuando sus estimaciones de costo y cardinalidad se alejan de la realidad. Este proyecto se justifica por los siguientes aportes:

- **Reducir los planes subóptimos:** el modelo aprende de los tiempos reales de ejecución para recomendar mejores órdenes de JOIN e índices.
- **Optimización externa sin sobrecarga:** al ejecutar el cómputo en un servidor de análisis independiente y conectarse a la base de datos solo para lecturas ligeras, se evita cargar el servidor de producción.
- **Portabilidad:** la capa de conexión desacoplada (MCP/adaptadores) permite analizar distintos servidores PostgreSQL remotos sin instalar software en ellos.
- **Uso eficiente de recursos:** la recomendación de índices considera el compromiso entre la mejora del workload y el espacio/mantenimiento.
- **Valor formativo:** integra Bases de Datos, SQL, algoritmos, Python, Machine Learning y arquitectura de software en un problema real y vigente en la investigación.

3.7 Objetivos

3.7.1 Objetivo General del Proyecto

Diseñar, desarrollar y evaluar un sistema externo y desacoplado basado en técnicas de Machine Learning supervisado para la optimización de consultas SQL en PostgreSQL, abordando la selección del orden de JOIN y la recomendación de índices.

3.7.2 Objetivos Específicos del Proyecto

1. Estudiar las técnicas de optimización de consultas de los SGBD y el estado del arte en Machine Learning aplicado a este problema (orden de JOIN, métodos de acceso e índices).
2. Construir un conjunto de datos de entrenamiento a partir de los benchmarks TPC-H y Join Order Benchmark (JOB), forzando el espacio de búsqueda de JOINS y extrayendo características de los planes `EXPLAIN (ANALYZE, FORMAT JSON)`.
3. Diseñar la arquitectura externa desacoplada (plataforma web en Docker, servidor de análisis en Python y capa de conexión de solo lectura/MCP al PostgreSQL remoto).
4. Desarrollar un modelo de Machine Learning (ranking/regresión) para recomendar el orden de ejecución de los JOIN, usando como variable objetivo el tiempo real de ejecución.
5. Desarrollar una estrategia de recomendación de índices a nivel de workload mediante índices hipotéticos (`HypoPG`) y evaluación de costo global.
6. Evaluar experimentalmente las estrategias sobre TPC-H/JOB, comparando los planes y tiempos recomendados con los del optimizador de PostgreSQL.
7. Analizar el compromiso entre rendimiento y recursos (mejora de tiempo frente a espacio y mantenimiento de índices) y verificar que el análisis externo no impacta la base de datos objetivo.

3.7.3 Objetivo General del Software

Proveer una plataforma web desacoplada, contenerizada en Docker, que capture los planes de ejecución de un SGBD relacional remoto (con validación en PostgreSQL) mediante un conector de solo lectura, ejecute los modelos de Machine Learning en un servidor de análisis independiente y presente en un dashboard: (i) la recomendación del orden de JOIN, (ii) el conjunto de índices más beneficioso para el workload y (iii) un diagnóstico de rendimiento que identifique las consultas que más sobrecargan la base de datos y sus posibles cuellos de botella, todo ello sin cargar la base de datos objetivo.

3.7.4 Descripción de las actividades para lograr los objetivos específicos

Objetivo Específico 1 (estudio del estado del arte):

- Revisar la bibliografía sobre optimización de consultas y Machine Learning para bases de datos (Neo, Bao, Lero, Balsa).
- Estudiar el optimizador de PostgreSQL y delimitar el alcance conceptual (ML supervisado vs. LLM vs. capa MCP).

Objetivo Específico 2 (dataset con TPC-H/JOB):

- Montar las bases de datos de benchmark TPC-H y Join Order Benchmark (JOB).
- Generar el dataset ejecutando el workload con `SET join_collapse_limit = 1` para forzar el espacio de JOINS, y capturar `EXPLAIN (ANALYZE, FORMAT JSON)` con sus features y el `Actual Total Time`.

Objetivo Específico 3 (arquitectura desacoplada):

- Diseñar e implementar la plataforma web en Docker y el servidor de análisis en Python.
- Implementar la capa de conexión de solo lectura (MCP/adaptadores) a PostgreSQL remoto.

Objetivo Específico 4 (modelo de orden de JOIN):

- Definir la representación y las características de consultas, relaciones y planes.
- Entrenar y ajustar el modelo de ranking/regresión (LightGBM/XGBoost).

Objetivo Específico 5 (recomendación de índices):

- Extraer atributos candidatos y evaluar índices hipotéticos con HypoPG.
- Generar recomendaciones considerando el costo global del workload y el espacio.

Objetivos Específicos 6 y 7 (evaluación y compromiso rendimiento/recursos):

- Comparar los planes y tiempos recomendados con los del optimizador de PostgreSQL sobre TPC-H/JOB.
- Analizar la reducción de tiempo frente al espacio/mantenimiento y verificar el no impacto sobre la base de datos objetivo.

3.8 Descripción de los Aspectos Fundamentales de la Metodología a utilizar

El proyecto combina una componente de investigación (estudio del estado del arte, generación del dataset y análisis experimental) con una de construcción de software (la plataforma web desacoplada). Como el mayor riesgo se concentra en el diseño y la evaluación de los modelos de Machine Learning —cuyo desempeño no se conoce de antemano—, se necesita una metodología que permita avanzar por partes, evaluar los resultados y corregir el rumbo de forma temprana. La siguiente tabla caracteriza el proyecto.

Ítem	Nivel	Descripción
Experiencia en el problema	Medio	Base en Bases de Datos, SQL y PostgreSQL; sin experiencia previa en optimización avanzada ni en ML aplicado.
Tamaño del problema	Grande	Estado del arte, generación del dataset, dos modelos y una plataforma web desacoplada.
Complejidad del problema	Alto	Orden de JOIN NP-duro; estimar tiempos reales con precisión es difícil.
Tamaño del software	Medio	Backend de análisis, capa de conexión (MCP) y dashboard web.
Complejidad del software	Alto	Integración con PostgreSQL, modelos de ML e infraestructura contenerizada.
Experiencia en el software	Bajo	Primera experiencia aplicando ML y una arquitectura desacoplada a un SGBD.
Modularidad	Alto	Componentes independientes: dataset, modelo de JOIN, índices, conexión y dashboard.

Considerando lo anterior, se adopta una **metodología de investigación cuantitativo-experimental aplicada**. Es la apropiada porque el objetivo central es *medir empíricamente*, mediante experimentos controlados y métricas cuantitativas (tiempos reales de ejecución), si los modelos de Machine Learning mejoran las decisiones del optimizador de PostgreSQL. Esta metodología se complementa con el marco **CRISP-DM** para el ciclo de datos y modelado, y con un desarrollo evolutivo por incrementos para la construcción de la plataforma web desacoplada.

El proyecto se organiza en las siguientes fases:

- **Fase 1 — Comprensión y formulación experimental:** delimitar el problema, las hipótesis y las variables (independientes: características del plan; dependiente: **Actual Total Time**), y definir el protocolo experimental.
- **Fase 2 — Preparación de los datos:** montar TPC-H y JOB, forzar el espacio de JOINs con `join_collapse_limit = 1`, y extraer las características desde los planes **EXPLAIN (ANALYZE, FORMAT JSON)**.
- **Fase 3 — Modelado predictivo:** entrenar y ajustar los modelos de ML (LightGBM/XGBoost) para predecir el tiempo de ejecución y rankear los planes candidatos.
- **Fase 4 — Evaluación experimental y validación estadística:** comparar las recomendaciones frente al optimizador de PostgreSQL sobre los benchmarks, midiendo la mejora de tiempo y su significancia.

- **Fase 5 — Despliegue de la arquitectura desacoplada:** integrar los modelos en la plataforma web contenerizada (Docker) con la capa de conexión de solo lectura (MCP).

El desarrollo de la plataforma se realiza de forma evolutiva por incrementos, revisando resultados y riesgos al término de cada fase.

3.9 Plan de trabajo a desarrollar

El proyecto se desarrolla a lo largo de dos semestres, siguiendo el calendario académico de la UBB: el 2º semestre 2026 (del 03 de agosto al 12 de diciembre de 2026) y el 1º semestre 2027. La Figura 1 presenta la carta Gantt y la tabla siguiente detalla la programación temporal.

Actividad	2026					2027				
	Ago	Sep	Oct	Nov	Dic	Mar	Abr	May	Jun	Jul
Estado del arte y aclaración conceptual										
Entorno y arquitectura desacoplada (Docker, MCP)										
Montaje de benchmarks (TPC-H / JOB)										
Generación del dataset (EXPLAIN ANALYZE)										
Análisis exploratorio y preparación de datos										
Modelo ML de orden de JOIN (LightGBM/XGBoost)										
Recomendación de índices (HypoPG)										
Integración en la plataforma web (MCP)										
Evaluación experimental (TPC-H/JOB vs PostgreSQL)										
Documentación e informe final										

Figura 1: Carta Gantt de las actividades del proyecto.

Programación Temporal

Nº	Actividad	Fecha inicio	Fecha final	Duración
1	Estudio del estado del arte y aclaración conceptual (ML/LLM/MCP)	04-08-2026	29-08-2026	4 sem.
2	Configuración del entorno y arquitectura desacoplada (Docker, MCP)	01-09-2026	26-09-2026	4 sem.
3	Montaje de los benchmarks TPC-H y JOB	29-09-2026	17-10-2026	3 sem.
4	Generación del dataset (forzar JOINS y EXPLAIN ANALYZE)	20-10-2026	21-11-2026	5 sem.
5	Análisis exploratorio y preparación de los datos	24-11-2026	12-12-2026	3 sem.
6	Modelo de ML para el orden de JOIN (LightGBM/XGBoost)	09-03-2027	10-04-2027	5 sem.
7	Estrategia de recomendación de índices (HypoPG)	13-04-2027	08-05-2027	4 sem.
8	Integración en la plataforma web desacoplada (MCP)	11-05-2027	29-05-2027	3 sem.
9	Evaluación experimental (TPC-H/JOB vs PostgreSQL)	01-06-2027	26-06-2027	4 sem.
10	Documentación e informe final	29-06-2027	17-07-2027	3 sem.

4 Referencias

- [1] R. Marcus et al., “Neo: A Learned Query Optimizer,” *PVLDB*, vol. 12, no. 11, pp. 1705–1718, 2019.
- [2] R. Marcus et al., “Bao: Making Learned Query Optimization Practical,” en *SIGMOD ’21*, pp. 1275–1288, 2021.
- [3] R. Zhu et al., “Lero: A Learning-to-Rank Query Optimizer,” *PVLDB*, vol. 16, no. 6, 2023.
- [4] Z. Yang et al., “Balsa: Learning a Query Optimizer Without Expert Demonstrations,” en *SIGMOD ’22*, 2022.
- [5] S. Chaudhuri y V. Narasayya, “An Efficient Cost-Driven Index Selection Tool for Microsoft SQL Server,” en *VLDB ’97*, pp. 146–155, 1997.
- [6] V. Leis et al., “How Good Are Query Optimizers, Really?” *PVLDB*, vol. 9, no. 3, pp. 204–215, 2015. (Join Order Benchmark)
- [7] Transaction Processing Performance Council, “TPC-H Benchmark.” <https://www.tpc.org/tpch/>
- [8] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” en *NeurIPS*, 2017.
- [9] T. Chen y C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” en *KDD ’16*, pp. 785–794, 2016.
- [10] G. Li, X. Zhou y L. Cao, “Machine Learning for Databases,” *PVLDB*, vol. 14, no. 12, pp. 3190–3193, 2021.
- [11] HypoPG — Hypothetical Indexes for PostgreSQL. <https://github.com/HypoPG/hypopg>

[12] PostgreSQL Global Development Group, “Documentación de PostgreSQL — EXPLAIN.” <https://www.postgresql.org/docs/current/sql-explain.html>

LA PRESENTE SOLICITUD DE INSCRIPCIÓN DE LA ACTIVIDAD DE TITULACIÓN SIGNIFICA UN COMPROMISO DE CUMPLIR LO ESTIPULADO EN ELLA.

Firma
Francisco Gonzalo Alexi Moya Alday

FECHA PRESENTACIÓN SOLICITUD: _____

RESOLUCIÓN DIRECTOR DE PROGRAMA/JEFE DE CARRERA	
APROBADO :	REPROBADO :
FECHA RESOLUCIÓN:	
_____ Firma DIRECTOR DE PROGRAMA/JEFE DE CARRERA	